Robust Failure, Conservative Repair: Textual Knowledge Distillation from Cross-Model Failures

Andrew Ren and Haokun Liu and Chenhao Tan
University of Chicago
{renqy, haokunliu, chenhao}@uchicago.edu

Abstract

Failure-based textual knowledge distillation aims to discover gaps in a model's knowledge by examining its task errors. The distilled knowledge can be useful for the reasoning of both this model ("source model") and other models. However, this transfer of knowledge may not be stable. We define a rule atom to be a standalone rule injected into a model's textual input at inference time. A rule atom can encode transferable task knowledge or model-specific reasoning patches that can confuse other models. Also, the injected rule atoms can be misapplied to unrelated cases, causing the model to incorrectly flip its answer based on irrelevant information. Building on a pipeline that distills training examples into task-specific cheat sheets that aid model reasoning, we examine when failure-derived rules can improve these cheat sheets. Our early experiment shows rule distillation from a single model's failures underperforms the baseline cheat sheet on non-source model families. This motivates *Robust Failure, Conservative Repair* (RFCR), a textual distillation procedure that derives rules from failures shared across models, sharpens their application boundaries using boundary cases, and abstains when no useful rule is found. On a 400-item BIG-Bench Hard (Suzgun et al., 2022) task set, RFCR improves the baseline cheat sheets from 68.50% to 71.25% (+2.75 pp; 95% CI [+1.25,+4.50]) without performance degradation on previously correct cases. Ablations and cross-model diagnostics support that accuracy gains come from both new knowledge injection and strict rule-application control.

1 Introduction

Large language models (LLMs) can turn concrete examples and task heuristics into textual artifacts. One example is a cheat sheet: a compact set of task rules distilled from sample task cases and provided at inference time. Cheat sheet in-context learning (CS-ICL) demonstrates this capability by compressing many-shot demonstrations into a reusable task-specific cheat sheet (Honda et al., 2025). Similarly, the SAIR Mathematics Distillation Challenge asks whether strong mathematical reasoning can be compressed into a compact, human-readable cheat sheet that helps a weaker model on equational-theory problems (SAIR Foundation, 2026). In both cases, we construct concise and readable cheat sheets that can efficiently transfer strong-model reasoning to cheaper models. A natural question is whether such useful artifacts can be distilled from model failures.

This question is a form of textual knowledge distillation. We focus on cases that the source model answers incorrectly when receiving a baseline cheat sheet. These failures may expose missing knowledge. For example, if a source model repeatedly errs on a geometric relation, a pronoun-resolution pattern, or a logical form, it likely lacks knowledge needed to answer such questions correctly. The difficulty is that source-model failures do not automatically produce transferable task knowledge. A failure-derived rule may encode a model-specific patch instead of transferable knowledge. Such a patch may correct one model's residual errors but hurt a target model that does not share those errors. Our Robust Failure Repair (RFR) diagnostic exhibits this behavior: source-only failure refinement underperforms the static CS-ICL baseline across all tested non-source model families.

We therefore introduce *Robust Failure, Conservative Repair* (RFCR), a conservative textual distillation pipeline that derives rule atoms from failures shared across models. We define a rule atom to be a standalone rule injected into the model's textual input at inference time. RFCR reframes failure-based refinement as *selective textual distillation*. A candidate rule atom should not be accepted merely because it fixes a source-model failure. It should be accepted only when its application boundary limits

Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing, pages 22806–22821
October 24-29, 2026 ©2026 Association for Computational Linguistics

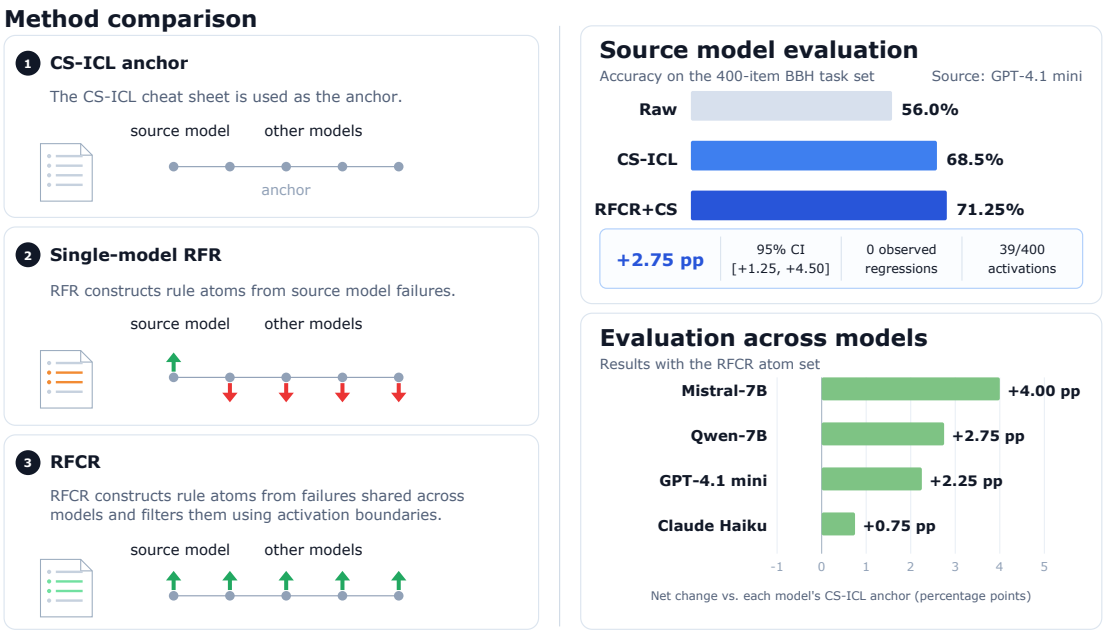


Figure 1: Overview of RFCR and its results. **Left:** CS-ICL provides the anchor cheat sheet. Source-only failure refinement can produce model-specific patches that improve the source model but hurt other models. RFCR constructs rule atoms from failures shared across models and constrains their activation using boundary cases. **Right:** On the 400-item BBH task set, RFCR improves CS-ICL by 11 items (+2.75 pp) without performance degradations on previously correct cases. These rule atoms also produce gains for the four models shown. Green and red arrows indicate improvements and performance degradations.

its use on unrelated or already-correct cases. To measure this behavior, we introduce *activation precision*. A rule atom is activated on a task case when the model uses it during reasoning. Activation precision is the proportion of cases on which the atom activates that belong to its intended target set. High activation precision means that a rule atom's activations align closely with the target cases it is designed to solve. Low activation precision, on the other hand, indicates that the rule activates on irrelevant cases where its encoded knowledge is not needed. The resulting procedure is conservative: RFCR accepts a rule atom only when the evidence supports cross-model accuracy gains without degrading performance on other cases.

In summary, we make the following contributions. First, we show that rule atoms distilled from one model's failures can correct the source model's errors but hurt other models that do not share the same errors. Second, we introduce activation precision to measure how frequently a rule atom is used on its intended cases rather than unrelated ones. Third, we develop *Robust Failure, Conservative Repair* (RFCR). This conservative pipeline derives rule atoms from shared failures, constrains their activation with boundary cases, and leaves the cheat sheet unchanged if none passes validation. On a 400-item BIG-Bench Hard (BBH) task set (Suzgun et al., 2022), RFCR improves CS-ICL by 11 items (+2.75 pp) without performance degradations on previously correct cases. More broadly, RFCR provides insight into a general conservative mechanism for failure-based knowledge distillation: find shared errors for rule construction, refine rule activation boundaries, and accept only rules that improve target accuracy without degrading other cases.

2 Failure-Derived Distillation Can Produce Model-Specific Patches

We first develop a Robust Failure Repair (RFR) diagnostic pipeline to explore how failure-based textual distillation can fail. Phase 0 builds a global cheat sheet from examples. Phase 1 patches the cheat sheet using source-model errors. Phase 2 adds local ACTIVATE-IF/WHY rules targeting residual source failures. Details of the RFR implementation can be found in Appendix D. As distillation moves from global examples toward residual source errors, the modification becomes more model-specific because it is refined on increasingly narrow error sets that other models may not share.

We then examine the impact of source-model-

Target model	CS-ICL	RFR	Δ
GPT-4.1	87.1	83.7	-3.4
Gemini-2.0	81.4	80.4	-1.0
Llama-3.3	78.3	75.7	-2.6
Claude-3.7	87.7	84.0	-3.7

Table 1: Early BBH diagnostic: mean accuracy over five non-ceiling tasks. The full source-refined RFR prompt underperforms CS-ICL on every non-source family.

specific error fixing on non-source models. On five non-ceiling BBH tasks, the full RFR pipeline falls below the static CS-ICL baseline for every non-source family (Table 1). This pattern shows that source-only refinement is not just hard to transfer, it can actively interfere with target models, which leads to performance degradation.

3 Transfer Theory for Failure-Derived Textual Knowledge Distillation

We define the anchor cheat sheet to be the baseline cheat sheet we aimed to improve on. An anchor-correct task case is a task case that is evaluated correctly under the anchor cheat sheet. Let C_0 be an anchor cheat sheet. For a model M, let $h_M^{C_0}(x)$ be its answer to input x under C_0. The anchor matters because the same atom may help under a raw prompt but harm on top of a stronger cheat sheet. We therefore define anchor-conditioned regions

$$F_M(C_0) = \{x : h_M^{C_0}(x) \neq y\}, \quad (1)$$

$$K_M(C_0) = \{x : h_M^{C_0}(x) = y\}. \quad (2)$$

A rule atom is a triple $m = (c, a_m, b_m)$: rule text c, a positive activation predicate $a_m(x)$, and an activation boundary predicate $b_m(x)$. Its effective activation is

$$\tilde{a}_m(x) = a_m(x)(1 - b_m(x)). \quad (3)$$

The rule-atom format can be found in Section 4.1, and at inference time both predicates reduce to keyword matching against the input question (Section 4.2).

Within the effective active region, three quantities determine the sign of the atom's effect:

$$\pi_M(m; C_0) = \Pr[x \in F_M(C_0) \mid \tilde{a}_m(x) = 1], \quad (4)$$

$$f_M(m; C_0) = \Pr[h_M^{C_0+m}(x) = y \mid \tilde{a}_m(x) = 1, x \in F_M(C_0)], \quad (5)$$

$$r_M(m; C_0) = \Pr[h_M^{C_0+m}(x) \neq y \mid \tilde{a}_m(x) = 1, x \in K_M(C_0)]. \quad (6)$$

Here $\pi_M(m; C_0)$ represents *activation precision*: among examples where the atom is used, how many are examples where the model needed help under the anchor? The terms f_M and r_M are conditional fix and performance degradation rates.

Proposition 1 (Activation-precision threshold)**.** *Assume* $\Pr[\tilde{a}_m(x) = 1] > 0$ *and that* m *does not change predictions outside its effective active region. The marginal accuracy effect of* m *on model* M *under anchor* C_0 *is positive if and only if*

$$\pi_M(m; C_0) > \frac{r_M(m; C_0)}{f_M(m; C_0) + r_M(m; C_0)}, \quad (7)$$

with zero effect when $f_M(m; C_0) = r_M(m; C_0) = 0$.

Proof. Only cases where m is effectively active can change answer. Conditional on activation, the fraction $\pi_M(m; C_0)$ are anchor failures and can be fixed with probability $f_M(m; C_0)$. The remaining fraction $1 - \pi_M(m; C_0)$ are anchor-correct examples and can be regressed with probability $r_M(m; C_0)$. Up to the positive factor $\Pr[\tilde{a}_m(x) = 1]$, the expected accuracy change is

$$\pi_M(m; C_0) f_M(m; C_0) - (1 - \pi_M(m; C_0)) r_M(m; C_0). \quad (8)$$

This quantity is positive exactly when the threshold above holds. □

This threshold explains why source gains do not imply transfer. A source-only gate can estimate whether m is useful for M_s, but transfer depends on whether the same atom activates precisely for a target M_t under the target anchor.

Proposition 2 (Source acceptance is not enough)**.** *For any* $f, r \in (0, 1]$*, there exist a source model* M_s*, a target model* M_t*, and a rule atom* m *such that* m *helps* M_s *and hurts* M_t*, even when both models have the same conditional fix rate* f *and performance degradation rate* r.

Proof. Choose $\pi_s > r/(f + r)$ and $\pi_t < r/(f + r)$. Construct active regions where the fraction of baseline failures is π_s for the source and π_t for the target, while both models share the same f and r. Proposition 1 gives a positive effect for M_s and a negative effect for M_t. □

Proposition 3 (Anchor-specific validity)**.** *A rule atom can have positive marginal gain under one anchor* C_0 *and negative marginal gain under another anchor* C_1 *for the same model* M.

Proof sketch. Choose anchors such that the atom's effective active region contains a high fraction of failures under C_0 and a low fraction of failures under C_1. The activation-precision threshold gives opposite signs. This formalizes why atoms selected on a raw prompt must be revalidated before being deployed on top of a stronger CS-ICL anchor. □

Proposition 4 (No-op safety)**.** *We define no-op as the deliberate choice of the pipeline to not proceed with any modification of the anchor cheat sheet. If inactive examples receive the anchor cheat sheet only, then a routed rule atom cannot change predictions outside its effective activation region.*

Proof. By construction, the inference procedure uses $h_M^{C_0}(x)$ whenever no selected atom has $\tilde{a}_m(x) = 1$. Thus non-activated examples are unchanged. If no atom activates, all examples follow the anchor. □

Failure overlap as a risk screen. Let F_s and F_t be source and target failure sets under C_0. We define the source-private failure region as the set of task cases where only the source model made a failed prediction and the shared failure region as the set of task cases where both source and target model give false answers. The shared failure region is $F_s \cap F_t$. The source-private failure region is $F_s \setminus F_t$. We summarize their overlap with

$$\rho_{s\to t} = \frac{|F_s \cap F_t|}{|F_s|}, \quad J(s,t) = \frac{|F_s \cap F_t|}{|F_s \cup F_t|}. \quad (9)$$

Low overlap warns that source-derived rules are likely to activate imprecisely for the target. High overlap is not a guarantee, because wording, routing, fix rate, and performance degradation risk still matter.

Lemma 1 (Overlap bound)**.** *Let* $\tilde{A}_m = \{x : \tilde{a}_m(x) = 1\}$ *be the effective active region of atom* m*. If* $\tilde{A}_m \subseteq F_s$*, then*

$$\pi_{M_t}(m; C_0) = \Pr[x \in F_t(C_0) \mid \tilde{a}_m(x) = 1] \leq \frac{|F_s \cap F_t|}{|\tilde{A}_m|}.$$

If $\tilde{A}_m = F_s$*, then* $\pi_{M_t}(m; C_0) = \rho_{s\to t}$.

Proof. Since $\tilde{A}_m \subseteq F_s$, we have $\tilde{A}_m \cap F_t \subseteq F_s \cap F_t$. Dividing by $|\tilde{A}_m|$ gives the bound. If $\tilde{A}_m = F_s$, the expression becomes $|F_s \cap F_t|/|F_s|$. □

4 *Robust Failure, Conservative Repair*

Robust Failure, Conservative Repair (RFCR) is a conservative textual knowledge distillation procedure. It treats failure-derived knowledge distillation as a routed repair procedure rather than as a global prompt update. We release our code and artifacts at https://github.com/ChicagoHAI/RFCR.

4.1 Rule Atom Construction

Let P be a proxy set of models used during refinement, excluding the held-out target family when such a target is defined. Under C_0, the set of validation examples V are divided into

$$V_{\text{shared}} = F_s \cap \left(\bigcup_{M_j \in P} F_j \right), \quad (10)$$

$$V_{\text{private}} = F_s \cap \left(\bigcap_{M_j \in P} K_j \right), \quad (11)$$

$$V_{\text{easy}} = K_s \cap \left(\bigcap_{M_j \in P} K_j \right). \quad (12)$$

RFCR generates candidate atoms from V_{shared}. It uses source-private failure examples (V_{private}) not as positive evidence but as boundary cases that tell the system when not to apply a rule. Easy examples (V_{easy}) are used to check for any rule-atom-induced performance degradation. The subset $V \setminus (V_{\text{shared}} \cup V_{\text{private}} \cup V_{\text{easy}})$ accounts for the cases where the source model provides correct reasoning but at least one proxy model fails. We put this subset aside intentionally as the source model's fix rate for cases in this set remains 0 regardless of the quality of candidates generated from this subset. This makes the conservative acceptance gate defined below too strict for any rule acceptance.

Rule-atom format. Each accepted rule atom is compact and human-readable:

> **RULE:** one task-level decision principle.
> **USE WHEN:** the minimal structural trigger.
> **DO NOT USE WHEN:** source-private or confounding cases.
> **CHECK:** one final verification step.

Atom generation. For rule atom generation, we divide the training split into two disjoint sets: a rule-generation set (60%) and a gate set (40%). Failure regions are computed and candidate atoms are generated on the rule-generation set only. The gate set is reserved for the acceptance test below, so no item is both used in rule generation and validation. We score the source and proxy panel models on the generation set using the baseline cheat sheet and group

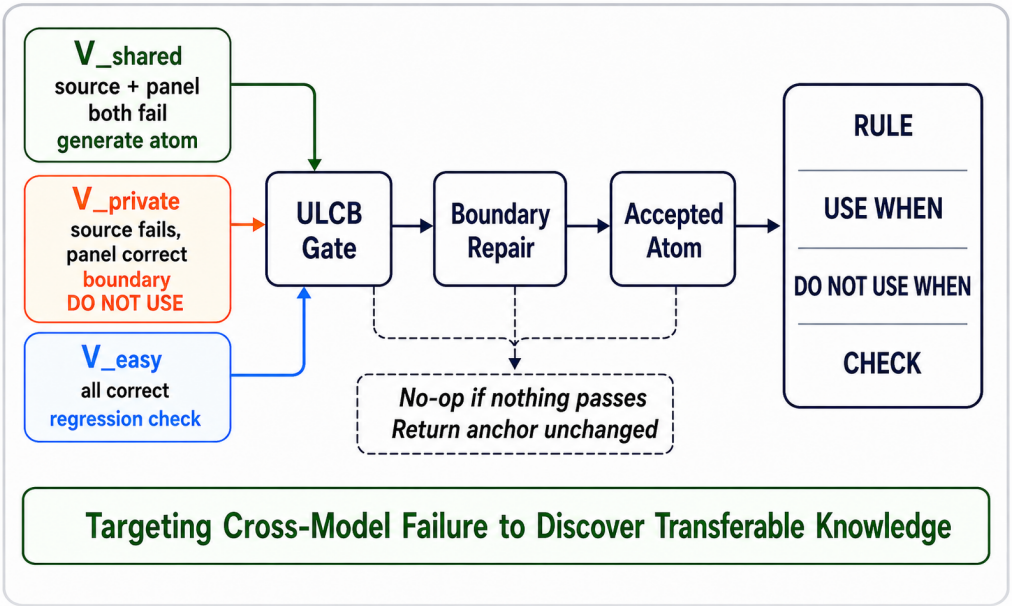


Figure 2: RFCR treats a failure-derived atom as rule text plus an activation predicate and an activation boundary predicate. If the gate fails or no route fires, the system returns the anchor output.

the shared failures by failure patterns. For each cluster, the source model receives the clustered failure examples together with source-private examples as explicit negative boundary cases, and is asked to write a rule in the four-field format above. Several candidates are generated per cluster at different temperatures, and we run each candidate through the validation set and decide acceptance through the acceptance gate. The full generation prompt is in Appendix B.

Conservative acceptance. Each candidate atom m is evaluated based on a conservative utility score:

$$U(m) = \min_{j \in P} \mathrm{LCB}_{95}[\Delta_j(V_s; m)] \tag{13}$$

$$- \lambda \max_{j \in P} \mathrm{UCB}_{95}[R_j(V_p; m)] \tag{14}$$

$$- \mu \max_{j \in P} \mathrm{UCB}_{95}[R_j(V_e; m)] - \nu|c|. \tag{15}$$

Here V_s, V_p, V_e are the shared, private, and easy regions defined above. $\Delta_j(V_s; m)$ is the fix rate of m on V_s. $R_j(V_p; m)$ and $R_j(V_e; m)$ are degradation rates on V_p and V_e. We use **Wilson score intervals** with z=1.96 for the confidence bounds and λ=μ=1.0 and ν=0.05 throughout the experiments (bound computation in Appendix B). The first term requires shared-failure improvement. The next two penalize performance degradations on source-private failures and easy examples. The length penalty discourages broad memories. A rule is accepted only when $U(m) > 0$. In the strict main-table deployment we also require positive net improvement and no observed performance degradations against CS-ICL-correct items.

Activation boundary repair and data separation. Candidate atoms can pass conservative validation while being incorrectly triggered on irrelevant cases. RFCR therefore includes a boundary-repair pass over activation predicates. This procedure fine-tunes the triggers using source-private failures with anchor-correct examples as negative controls. It is worth noting that the pass changes only whether an accepted atom is allowed to fire, but not the content of the atom. We first conduct a trial run on the items with the accepted atoms active to observe which items are fixed or degraded and then modify the trigger accordingly before freezing them for the final rerun. Since the repair procedure utilizes the test items, the 400-item result should be read as a controlled development result rather than a held-out test. We therefore report the test result before and after repair as an ablation result. For held-out validation, we report the frozen knowledge atom set's performance on 295 official BIG-Bench items unseen during the rule atom generation and boundary repair phases.

4.2 Rule Atom Routing at Inference

At inference time, the model receives C_0 plus the atoms whose activation condition fires and whose activation boundary condition does not fire on the

task case. If no atom activates, the input is exactly the unchanged CS-ICL anchor. Atom activation is determined by keyword matching between the activation predicates (USE WHEN / DO NOT USE WHEN section) of the rule atom and task case question text. We define this mode as routing in comparison to the global mode. Under global mode, all accepted atoms are presented to each task case with activation decision relies entirely on LLM's interpretation of the rule's activation predicates at inference time.

No-op deployment. If no atom passes route and activation boundary checks, RFCR returns exactly the CS-ICL output. Abstention is part of our method: the system should not add a risky rule merely because a source model improved during development.

5 Experiments and Results

Evaluation Task Sets. The main evaluation uses BBH tasks for which the RFCR pipeline produced main-eligible routed atoms: disambiguation QA, formal fallacies, geometric shapes, and object counting. This main task set contains 400 examples. We also report a supplementary 18-task BBH task set as a sanity check on scale and a 295 official BIG-Bench upstream examples outside the seen train and test BBH subsets for validation. The 295 held-out set consists of 8 disambiguation QA, 100 formal fallacies, 87 geometric shapes, and 100 object-counting examples. The selection of these examples did not rely on any prior knowledge of the performance or expected outcomes of either our method or the baselines.

Anchor and protocol. The anchor is a GPT-4.1-mini-generated CS-ICL (Honda et al., 2025) cheat sheet, scored by GPT-4.1-mini in the unified evaluator. For every inactive item, the RFCR condition copies the CS-ICL output exactly. The strict rerun uses the same task set, candidate rule atoms, prompt builder, parser, retry path, scoring guard, low-concurrency call schedule, and 10,000 paired-bootstrap samples across conditions.

Models and access. We choose `gpt-4.1-mini` as our source model. This model is used in the generation of the CS-ICL baseline cheat sheets as well as all candidate atoms generation. The proxy models we choose are `qwen2.5-7b-instruct` and `mistral-7b-instruct-v0.3` with approximately 7B parameters each. These models are hosted through vLLM on a compute cluster. Models used for cross-model transfer evaluation (`claude-haiku-4.5`, `gemini-2.5-flash-lite`) and models used for the early RFR diagnosis (GPT-4.1, Gemini-2.0-Flash, Llama-3.3-70B, Claude-3.7-Sonnet) are accessed through OpenRouter.

Computational budget. The RFCR pipeline using CS-ICL cheat sheet as baseline takes around 35,000 to 45,000 model calls. In our case since we are hosting two proxy models through vLLM, the pipeline uses around 25,000 paid API calls, which costs around 20 USD. The two proxy models took 48 GPU hours in total. We also report the per stage token usage. Each CS-ICL cheat sheet generation process consumes 149,683 input tokens and 9,122 output tokens. Each evaluation pass of the 400-item set takes 955,275 input tokens and 1,290 output tokens.

Main controlled result. Table 2 gives the strict source-protocol result. On the 400-item BBH task set, RFCR improves the CS-ICL baseline from 68.50% to 71.25%, a gain of +11 items or +2.75 pp, with a 95% confidence interval of [+1.25,+4.50] pp and no paired performance degradations.

Regarding held-out confirmations, we conducted an additional evaluation using the same frozen knowledge atom set constructed for the main result (plus the CS-ICL baseline). We use the same evaluation pipeline as in the main result for consistency. Out of the 295 test cases, we observed 16 cases with atom activation, while the other 279 were CS-ICL passthrough. On accuracy results, we see that the CS-ICL baseline scored 66.10%, and our overlay result gave 67.46%. This +1.36 pp improvement results from four fixes and zero observed regressions.

Full 18-task breakdown. We also report the result of RFCR on the 18-task BBH set (Table 11 in Appendix E). The supplementary 18-task check improves from 75.58% to 76.22%, also without performance degradations (Table 2). We observe that for 14 out of the 18 tasks RFCR refrains from adding any atomic rules upon the CS-ICL baseline. The three tasks that RFCR does generate valid atomic rules are formal fallacies, disambiguation QA, and geometric shapes, which constitute the task sets we use for our main results. The object counting task is also included in the main task set.

Task Set Evaluated	n	Raw	CS-ICL	RFCR	Δ	95% CI	Reg.
BBH main eligible atom tasks	400	56.00	68.50	71.25	+2.75	[+1.25,+4.50]	0
BBH held-out cases	295	–	66.10	67.46	+1.36	[+0.34,+2.71]	0
BBH full18, supplementary	1716	70.75	75.58	76.22	+0.64	[+0.29,+1.05]	0

Table 2: Strict unified-evaluator results after disambiguation activation boundary repair. Δ is absolute accuracy improvement in percentage points over the GPT-4.1-mini CS-ICL anchor. The full18 row is supplementary because most tasks are inactive passthrough.

Model	n	CS-ICL	RFCR+CS	Fix	Reg.	Net	Δ pp / 95% CI
`anthropic/claude-haiku-4.5`	400	75.75	76.50	4	1	+3	+0.75 [-0.25, 2.00]
`google/gemini-2.5-flash-lite`	400	64.25	63.75	3	5	-2	-0.50 [-2.00, 1.00]
`mistral-7b-instruct-v0.3`	400	37.25	41.25	16	0	+16	+4.00 [2.25, 6.00]
`qwen2.5-7b-instruct`	400	51.00	53.75	13	2	+11	+2.75 [1.00, 4.75]

Table 3: Cross-model rerun with the repaired atom set. Rule atoms are applied without target-model route tuning (i.e. visible during all task case evaluations with soft activation boundary check by LLM at inference time). The evidence supports positive-net transfer on several families, but not a cross-model zero-degradation guarantee.

Method	Acc.	Δ	95% CI	Fix	Reg.
CS-ICL anchor (seed)	68.50	–	–	–	–
ProTeGi	70.50	+2.00	[−1.50,+5.50]	32	24
GEPA	65.50	−3.00	[−6.25,+0.25]	16	28
RFCR	**71.25**	**+2.75**	[+1.25,+4.50]	11	**0**

Table 4: Prompt-optimization baselines on the same 400-item main task set. ProTeGi and GEPA start from the exact CS-ICL frozen cheat sheet as we used for RFCR.

Task	CS-ICL	RFCR+CS	Act.	Fix	Reg.	Net
Disambig. QA	83	87	15	4	0	+4
Formal fallacies	67	67	4	0	0	0
Geometric shapes	56	63	20	7	0	+7
Object counting	68	68	0	0	0	0
Total	274	285	39	11	0	+11

Table 5: Task-level decomposition on the 400-item main task set. Counts are out of 100 items per task. "Act." is the number of activated routed-rule applications.

However, its candidate atom was rejected by the acceptance gate, so it runs as passthrough of the CS-ICL baseline. It is worth noting that the 18-task run is a sanity check for inactive passthrough rather than a benchmark-wide improvement claim.

Prompt-optimization baselines. To evaluate RFCR as a prompt optimization method, we ran two prompt optimization methods, ProTeGi (Pryzant et al., 2023) and GEPA (Agrawal et al., 2025). Both baselines use the same training and testing sets as in our RFCR main results. Both methods use gpt-4.1-mini as the source model and initiate with the same CS-ICL cheat sheet as RFCR. We use the default parameters of both methods. The results show that among the 400 test examples, ProTeGi achieved 32 fixes with 24 regressions. GEPA got 16 fixes but 28 regressions. Overall, we see that these optimization methods yielded more fixes than our RFCR pipeline at the cost of causing many regressions along the way. In GEPA's case, the revision-induced regression outnumbered the fixes it introduced, making it perform worse than the CS-ICL baseline. This result echoes our concern about the uncertainty of prompt optimization's effects on already correct cases and emphasizes the value of the strict knowledge atom selection pipeline proposed by RFCR. More details of the baseline method execution are in Appendix C.

Cross-model diagnostics. Table 3 evaluates the repaired atom set on model families. The repaired atoms have positive net transfer on Claude Haiku, Mistral, and Qwen. We further observe that models with relatively weaker reasoning capability, such as Qwen and Mistral, show larger net gains with relatively low performance degradation. This supports our interpretation that RFCR distills task-specific knowledge that transfers broadly rather than merely patching source-model errors. However, Gemini Flash Lite has negative net transfer and several models show nonzero performance degradations, indicating that even task-specific rules may not universally apply across all model families.

Task-level decomposition. Table 5 shows that the gain is sparse. Geometric shapes contributes +7 items and disambiguation QA contributes +4. Formal fallacies is neutral; object counting is a no-op under the repaired CS-ICL anchor. This sparse pattern is consistent with our methodology: RFCR improves only where a routed repair is available and otherwise leaves the anchor unchanged.

Version	n	Activated	Fix	Reg.	Net	Acc.	Δ vs CS
Original locked package	400	59	9	0	+9	70.75	+2.25
Repaired route	400	39	11	0	+11	71.25	+2.75

Disambiguation route version	Activated	Fix	Reg.	Net
Original broad route	35	1	0	+1
Repaired narrow route: `needed_one OR told_that`	15	4	0	+4

Table 6: CS-ICL-anchored same-cache route ablation. The repaired activation trigger here is Disambiguation-QA-specific and presented as an example of boundary repair rather than a generalizable mechanism.

Condition	n	Acc.	Correct	Fix	Deg.	Net	Activated
No atom	400	56.00	224	0	0	0	0
Global atom	400	66.00	264	78	38	+40	400
Raw-Routed atom	400	66.00	264	43	3	+40	132
Boundary-routed atom	400	69.50	278	54	0	+54	132

Table 7: Rule Atom Activation Control Ablation with no Baseline Cheat Sheet

Boundary-repair ablation. Because the main result uses boundary repaired routing for Disambiguation QA items, we investigate exactly how this boundary repair effort helps in test accuracy improvement. Table 6 compares the original locked atom set with the boundary repaired set with the same CS-ICL baseline and test set. The repaired route improves the test accuracy to 71.25% without performance degradations. The boundary-repaired routing reduces the number of activations on test cases from 35 to 15 but increases the number of fixes from 1 to 4. The result shows that narrowing the activation space improves model performance by directing each atom only to cases where its encoded knowledge is relevant. This supports the activation-precision view: sharpening a rule's activation boundary can increase fixes without increasing performance degradations.

Global versus routed exposure. We also examine the effect of different activation control measures on the same kinds of atoms when baseline cheat sheets are not presented. Table 7 shows that global exposure leads to high activation frequency of the atoms but also leads to a substantial performance degradation. The raw-routing mode (LLM receives the full atom list at inference time, but task results of cases not activating any atoms fall back to the no atom result) reduces the degradation rate drastically. Boundary-routed mode (what standard RFCR uses) further improves the accuracy result by boosting activated task case accuracy and eliminating degradation.

Generated rules versus activation boundary repair. Table 8 separates candidate generation from conservative validation. Generation-only approach fixes more cases but introduces five paired degradations. The upper/lower confidence bounds (ULCB) filtering show no performance degradations with the same net accuracy gain. The repaired activation boundary approach further improves on both net gain and activation precision. This matches our claim that the main result comes from sparse conservative routing instead of indiscriminately adding more generated text.

Variant	Acc.	Fix	Reg.	Net	Act.
CS anchor	68.50	0	0	0	0
Gen. only	70.75	14	5	+9	79
Gen.+ULCB	70.75	9	0	+9	59
+Boundary	71.25	11	0	+11	39

Table 8: Generated-only versus activation-boundary-repaired atoms.

6 Analysis

Sparse gains. The main gain of RFCR is sparse and routed across subsets of cases with similar demands for specific knowledge, rather than a global marginal improvement over the entire task. Only 39 item-rule applications activate on the 400-item task set, yet they account for +11 net fixes without any source-model performance degradations. This aligns with our Proposition 1: a valuable atom needs high activation precision on the subset where it fires, not coverage of every item.

Activation precision. Table 9 reports atom-level activation precision and net gain. Fix precision here refers to the number of fixed cases among all activated cases, while activation precision measures the fraction of CS-ICL failures among activated cases. The geometric atom has both high activation

Atom	Act.	Fix. prec.	Act. prec.	Fix	Net
Disambig-v1.2	15	26.7	40.0	4	+4
FF-neg.	4	0.0	50.0	0	0
Geo-1	8	12.5	12.5	1	+1
Geo-6	12	50.0	91.7	6	+6

Table 9: Activation-precision diagnostics.

precision and the largest net gain. The formal-fallacies atom is safe but so underactive that it does not sway the test result either way. We see here that fix precision and activation precision are both positively correlated with the net gain, while the total number of activated cases does not.

Abstention is part of the method. Object counting has eligible task coverage but zero active applications in the repaired main table. This is a no-op decision made by the pipeline rather than a failure to deploy. The supplementary full-18 result also shows that inactive passthrough does not introduce any performance degradation.

Cross-model performance degradations are informative. Cross-model performance degradations show the weakness of source-derived activation boundary. The same atom can have positive net transfer and still regress target-correct items if the target activation boundary differs from the source activation boundary. This motivates leave-one-family-out proxy gating and per-target activation boundary diagnostics.

7 Related Work

Textual knowledge distillation and cheat sheets. CS-ICL distills many-shot in-context examples into a compact textual summary, reducing inference-time prompt cost while preserving much of the benefit of long demonstrations (Honda et al., 2025). Dynamic Cheatsheet studies persistent, evolving memory for black-box language models at test time (Suzgun et al., 2026). Our work is complementary: rather than asking only whether a distilled rule is useful in aggregate, we ask when failure-derived textual knowledge transfers across model families without becoming a model-specific patch.

Prompt optimization and reflective search. Prompt optimization methods use data, feedback, search, evolution, or textual gradients to improve model inputs and behavior (Pryzant et al., 2023; Fernando et al., 2023; Yang et al., 2024; Khattab et al., 2024; Yuksekgonul et al., 2025; Agrawal et al., 2025). These methods usually optimize a prompt for a measured development distribution. RFCR focuses on a narrower distillation risk: a failure-derived atom can be accepted by the source model yet hurt another model if it activates on examples that the target already solves.

Reasoning benchmarks. We evaluate on BIG-Bench Hard (BBH), a challenging subset of BIG-Bench designed to stress reasoning behavior (Srivastava et al., 2022; Suzgun et al., 2022). Because several BBH tasks are sensitive to prompt wording and reasoning format, they are useful for testing whether distilled textual repairs are robust rather than source-specific. We discuss AGIEval (Zhong et al., 2024) as a diagnostic benchmark, but do not use it as a main claim until a locked nonnegative-anchor protocol is available.

8 Conclusion

Rules distilled from one model's failures can improve source model's accuracy but hurt other models' performance when applied to unrelated or already-correct cases. We use activation precision to measure how often a rule is applied to the cases it was designed to address. RFCR turns this analysis into a procedure by constructing rule atoms from shared failures, refining activation boundaries with source-private failures and anchor-correct examples, and applying conservative acceptance checks to candidate rule atoms. On a 400-item BBH task set, RFCR improves the CS-ICL baseline accuracy by 2.75 percentage points without degrading any previously correct case. The success of RFCR suggests that improving cheat sheets through textual knowledge distillation requires both extracting new knowledge and controlling when that knowledge is applied. Future work should explore forms of cheat sheet modification beyond simply appending rules to separate the effects of cheat sheet structure from those of newly distilled knowledge.

Ethical Considerations

This work studies textual distillation on reasoning benchmarks. The main risk is overclaiming reliability: a rule artifact that improves one model may harm another. We therefore report paired performance degradations, preserve a no-op fallback, and emphasize that source-model gains alone are not evidence of general improvement. The experiments do not require personal data.

Limitations

The main improvement is modest and concentrated in geometric shapes and disambiguation QA. The evidence supports conservative routed textual distillation as a sparse no-degradation improvement over a strong same-generator CS-ICL anchor instead of a broad benchmark optimization claim.

The zero-degradation property holds only for the locked source-model protocol. Cross-model performance degradations remain nonzero for some targets, reflecting that activation predicates were derived from source-model failures and not validated against every target-model activation boundary. The paper includes a retrospective leave-one-family-out proxy-gating diagnostic, but it does not yet regenerate atoms with each family fully excluded. A stronger test should perform end-to-end leave-one-family-out generation, gating, freezing, and target-only evaluation.

The main evaluation task set contains four BBH tasks with eligible routed atoms. Broader benchmark-scale claims require more tasks, learned routers, and stronger held-out validation. The supplementary full-18 result is useful as a sanity check, but it should not be interpreted as evidence that every task benefits from RFCR, because most items use inactive passthrough.

The main 400-item result is a controlled development result, not a fresh held-out test after activation boundary repair. The final disambiguation activation boundary was repaired using item-effect evidence from the atom-enabled pilot before the repaired route was frozen and rerun under a strict evaluator.

Acknowledgments

We thank the anonymous reviewers for their insightful comments. We also thank the NeuriCo project from Chicago Human+AI Lab for its assistance in early-stage mechanistic exploration. This work was supported in part by compute credits from Modal, provided through CMSC 25750/35750 Large Language Models at the University of Chicago, as well as the University of Chicago AI initiatives.

References

Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2025. Gepa: Reflective prompt evolution can outperform reinforcement learning. *Preprint*, arXiv:2507.19457.

Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. 2023. Promptbreeder: Self-referential self-improvement via prompt evolution. *Preprint*, arXiv:2309.16797.

Ukyo Honda, Soichiro Murakami, and Peinan Zhang. 2025. Distilling many-shot in-context learning into a cheat sheet. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 17158–17178, Suzhou, China. Association for Computational Linguistics.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. Dspy: Compiling declarative language model calls into self-improving pipelines. In *The Twelfth International Conference on Learning Representations*.

Reid Pryzant, Dan Iter, Jerry Li, Yin Lee, Chenguang Zhu, and Michael Zeng. 2023. Automatic prompt optimization with "gradient descent" and beam search. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 7957–7968, Singapore. Association for Computational Linguistics.

SAIR Foundation. 2026. Mathematics distillation challenge: Equational theories, stage 1. https://competition.sair.foundation/competitions/mathematics-distillation-challenge-equational-theories-stage1/overview. Accessed 26 August 2026.

Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, Abu Awal Md Shoeb, Abubakar Abid, Adam Fisch, Adam R Brown, Adam Santoro, Aditya Gupta, Adrià Garriga-Alonso, and 1 others. 2022. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *arXiv preprint arXiv:2206.04615*.

Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V Le, Ed H Chi, Denny Zhou, and Jason Wei. 2022. Challenging big-bench tasks and whether chain-of-thought can solve them. *arXiv preprint arXiv:2210.09261*.

Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. 2026. Dynamic cheatsheet: Test-time learning with adaptive memory. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7080–7106, Rabat, Morocco. Association for Computational Linguistics.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. Large language models as optimizers. *Preprint*, arXiv:2309.03409.

Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. 2025. Optimizing generative ai by backpropagating language model feedback. *Nature*, 639:609–616.

Wanjun Zhong, Ruixiang Cui, Yiduo Guo, Yaobo Liang, Shuai Lu, Yanlin Wang, Amin Saied, Weizhu Chen, and Nan Duan. 2024. AGIEval: A human-centric benchmark for evaluating foundation models. In *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 2299–2314, Mexico City, Mexico. Association for Computational Linguistics.

A Reproducibility Statement

We release our code, the frozen knowledge atom set, and the CS-ICL cheat sheets used in all experiments at https://github.com/ChicagoHAI/RFCR. The final run uses a fixed task set, fixed CS-ICL cache, fixed selected-atom file, the same parser and prompt builder across conditions, a scoring guard, low-concurrency calls, inactive passthrough for non-activated items, and a paired bootstrap with 10,000 samples. The reported CS-ICL column is the unified-evaluator GPT-4.1-mini CS-ICL anchor/cache condition and should not be mixed with separately reported deployment numbers from earlier runs. The main 400-item result and all ablation tables are from a single locked run. The exploratory holistic rewrite table (Appendix F) reports means over three seeds (1000, 2000, 3000). Cross-model 95% CIs are computed per model using the same paired-bootstrap procedure.

B RFCR Implementation Details

Shared-failure identification. A failure is classified as *shared* if at least one proxy model $M_j \in P$ also fails the item under C_0. This is a union criterion: $V_{\text{shared}} = F_s \cap (\bigcup_j F_j)$. A failure is *source-private* if *every* panel model gets it right: $V_{\text{private}} = F_s \cap (\bigcap_j K_j)$. Items that all models including the source get right form V_{easy}. Cluster size within V_{shared} serves as a soft priority signal for which failure patterns to write atoms for first.

Atom generation prompt. Atoms are generated by presenting the source model with a curated batch of shared-failure examples together with a selection of source-private boundary examples, then issuing the following template:

```
You are given examples where a
language model failed despite
having a general cheat sheet.
Identify the common structural
pattern and write a repair rule.
Failed examples (model got
wrong):
[SHARED_FAILURE_EXAMPLES]
Boundary examples (model gets
right -- the rule must NOT fire
here):
[SOURCE_PRIVATE_EXAMPLES]
Write the rule in this exact
format:
RULE: <one task-level decision
principle>
USE WHEN: <minimal structural
trigger>
DO NOT USE WHEN: <conditions
from the boundary examples>
CHECK: <one final verification
step>
Be as specific as possible.  Do
not write a rule that applies to
all task examples.
```

The **DO NOT USE WHEN** field is seeded from V_{private} so the model is nudged toward writing a narrow activation boundary condition rather than a broad topic match. Multiple candidates are drawn per cluster using temperature sampling; all candidates are then scored by the ULCB gate.

Upper/Lower confidence bounds. Confidence bounds are Wilson score intervals at z=1.96. $\text{LCB}_{95}[\Delta_j]$ is the Wilson lower bound of the observed fix proportion on the gate subset, and $\text{UCB}_{95}[R_j]$ is the Wilson upper bound of the observed performance degradation proportion. An atom passes the gate only when $U(m) > 0$. The strict deployment condition additionally requires zero *observed* performance degradations against CS-ICL-correct items across all panel models (not just a UCB bound).

Activation Boundary repair prompt. After an atom passes the ULCB gate, its activation predicate is inspected for over-triggering. The repair prompt presents the source model with over-triggered cases and asks for a narrower condition:

```
An accepted rule fires on
the following examples but
should not (anchor-correct or
source-private cases):
[OVER_TRIGGERED_EXAMPLES]
The rule correctly fires on
these examples:
[CORRECTLY_TRIGGERED_EXAMPLES]
```

```
Current USE WHEN:
[CURRENT_TRIGGER]
Write a narrower USE WHEN
condition that still covers
the correct examples and
excludes the over-triggered
ones.  Express it as a minimal
keyword or structural pattern
(e.g., "contains X OR contains
Y").
Narrowed USE WHEN:
```

Each candidate narrowing is evaluated on the locked task set. The narrowing that maximizes net gain while maintaining zero observed performance degradations is selected.

Scoring guard and retry logic. Each model call goes through a scoring guard that checks whether the returned answer can be parsed to a valid option or numeric value. Failed parses trigger up to two retries with a slightly relaxed extraction prompt before the item is marked as a parse failure. Items with parse failures are excluded from paired comparisons but logged. The low-concurrency call schedule (sequential rather than batch-parallel) was adopted to reduce inter-call variance across conditions; all conditions for a given item are scored in the same session where possible.

Paired bootstrap. The 95% CI and performance degradation counts are computed over 10,000 paired bootstrap resamples of the item-level effect vector $e_i = \mathbf{1}[\text{RFCR correct}] - \mathbf{1}[\text{CS-ICL correct}]$. A performance degradation is defined as $e_i = -1$; a fix as $e_i = +1$; a no-op as $e_i = 0$. The reported CI is the percentile interval of the bootstrap distribution of $\sum_i e_i/n$.

C Baseline Method Execution Details

Shared protocol. Both baselines follow the same seeded, leakage-controlled protocol. The seed prompt is the locked gpt-4.1-mini CS-ICL cheat sheet for each task, i.e., the same anchor RFCR refines. Optimization touches only the 150-item train split per task; every optimization-time prompt and response is logged and scanned by an automated audit for verbatim test-item text, with zero matches on all four tasks for both methods. The single best prompt per task is frozen (SHA-256 recorded) before any test call, then evaluated once on the task's 100 test items through the locked evaluator path (temperature 0.0, seed 42, `max_tokens` 64, gpt-4.1-mini, unchanged prompt wrapper and answer parser). Per-item fixes and regressions are computed against the same frozen CS-ICL anchor cache as the main result, and pooled deltas use a task-stratified paired bootstrap with 10,000 resamples. The verdict parse rate is 100% (400/400) for both methods.

ProTeGi. We run the LMOps `prompt_optimization` implementation of ProTeGi (textual-gradient beam search) with its documented default configuration: 4 optimization rounds, beam size 4, minibatch size 32, 4 gradients per round with 4 errors per gradient, up to 8-fold prompt expansion, and the UCB bandit evaluator (8 evaluation rounds, 8 prompts per round, 32 samples per evaluation, c=1.0). `gpt-4.1-mini` serves as both the task model and the gradient model; optimization decoding uses temperature 0.0 with a fixed run seed. The candidate that ProTeGi mutates is the cheat-sheet text itself, and every optimization-time scoring call wraps the candidate in the locked evaluator template, so optimization observes the exact deployment presentation. Because the stock implementation selects its final prompt by test-set score, we modified final selection to use full-train accuracy over the final beam; no test items are read anywhere in the optimization loop. Optimization used 28,816 model calls across the four tasks. On disambiguation QA the beam search retained the seed cheat sheet unchanged (hash-identical) after all rounds.

GEPA. We run the standalone `gepa` library (v0.1.1) through its public `optimize()` API with its default adapter, supplying the seed cheat sheet as the system-prompt candidate. The 150 train items are shuffled once with a fixed seed and split 75/75 into the library's internal train and validation sets; the metric is exact-match correctness through our own task parsers. Because the library requires an explicit budget, we set `max_metric_calls` to 680 per task, the value produced by the DSPy GEPA "light" preset budget formula; all other settings are library defaults, including default sampling on optimization-time calls. `gpt-4.1-mini` serves as both the task model and the reflection model. GEPA returned the seed prompt unchanged on three of four tasks (every reflective mutation scored at or below the seed on its own validation split), so its test-time deltas on those tasks measure fresh-call variance against the cached anchor rather than a prompt change; its one adopted rewrite (object counting)

yielded 11 fixes and 12 regressions.

D RFR Diagnostic Pipeline

The RFR pipeline is the source-only refinement baseline we used to produce the diagnostic results in Table 1. It runs in three phases on a source model over a training split, with each phase accepting patches only when they pass source-model quality gates with no multi-model evaluation at any stage. This is the fundamental reason its output becomes model-specific. Below we describe each phase in detail.

Phase 0: Bootstrap cheat sheet. The pipeline samples the first 75 training examples (configurable). For examples lacking an explicit chain-of-thought, it first generates zero-shot reasoning traces via the source model. The examples are then formatted as Question / Reasoning / Answer triplets and passed to the following prompt:

```
Create a cheat sheet based on
the examples below.  You will
be asked to answer questions
similar to these examples during
the test, without being allowed
to refer to the examples at
that time.  Your task here is
to make a cheat sheet that will
help you answer such problems
correctly.  First, carefully
read the examples below and
identify which ones you find
most difficult to answer.
[EXAMPLES]
Now, create a cheat sheet to
help you solve the difficult
examples.  Exclude any
content that is easy for you,
and only include specific,
detailed points to address the
challenging ones.
```

The output is a `prior_knowledge` text block that forms the starting anchor for Phase 1.

Phase 1: Iterative prior-knowledge patching. Phase 1 runs an iterative repair loop over source-model failures under the current `prior_knowledge`. Each iteration (up to a configurable maximum, default 4) samples up to 15 failures and constructs a *failure block* for each: the input, the expected and predicted answers, the source model's wrong reasoning trace, and — when oracle traces are available — the correct reasoning from a stronger reference model. The patch prompt is:

```
You are refining a knowledge
guide that helps a model answer
questions correctly.
=== CURRENT KNOWLEDGE GUIDE ===
[PRIOR_KNOWLEDGE]
=== END KNOWLEDGE GUIDE ===
The model is making the
following errors:
[FAILURE_BLOCKS]
Produce an IMPROVED version of
the knowledge guide that helps
avoid these mistakes.  You may:
- ADD a new rule, clarification,
or concrete example
- MODIFY an existing rule to be
more precise or correct
- REMOVE a rule that is actively
causing errors
Requirements:  preserve rules
that are working correctly;
focus on ABSTRACT REASONING
PRINCIPLES, not memorization of
specific examples; output ONLY
the improved knowledge guide
text.
```

A candidate patch is committed only when it passes two source-model gates: (a) it fixes at least 20% of the sampled failures (*fix-rate gate*), and (b) it does not regress more than 20% of a sample of currently-correct items (*performance degradation gate*). The loop exits when source accuracy reaches 85% or when two consecutive iterations produce no accepted patch. The oracle contrast — showing the correct reasoning from a stronger model alongside the wrong reasoning — provides a concrete target for the patcher, but is itself derived from source-model failures and carries no cross-model validity signal.

Phase 2: Case study generation. Phase 2 operates on residual failures after Phase 1. It partitions failures by structural feature signature (e.g., argument form, entity type, geometric relation) and, for each partition with at least 3 failures, generates up to 3 candidate case studies using a task-specific structured prompt. Each generated case study has the following fields:

```
=== CASE STUDY: [title] ===
FAILURE_TYPE: A (missing
knowledge) | B (wrong pattern)
ACTIVATE IF:
- [structural condition 1]
- [structural condition 2]
DO NOT ACTIVATE IF: [boundary
condition]
COMMON WRONG MOVE: [what the
model typically does wrong]
NEXT CHECK: [mechanical
verification step]
WHY THIS WORKS: [1-2 sentences
explaining the fix]
SUPPORT:
```

```
• E1 = ...   | E2 = ...   |
Answer:  ...   -- [note]
```

The ACTIVATE-IF conditions become the activation predicate; DO NOT ACTIVATE IF becomes the boundary predicate. A candidate case study is accepted when it (a) fixes at least 30% of the partition failures, (b) does not regress more than 15% of correct items, and (c) is not a duplicate or near-duplicate of an existing case study through an LLM judge. Accepted case studies are appended to the cheat sheet and re-scored; partitions are refreshed after each flush, and the loop runs for up to 5 iterations.

Why model-specificity accumulates. All three gates in Phases 1 and 2 are evaluated against the *source model only*. A Phase 1 patch is accepted if it fixes source failures and does not break source-correct items. A Phase 2 case study is accepted if its partition fix rate and performance degradation rate are satisfactory on the source. Neither the fix-rate threshold nor the degradation threshold is measured against any other model other than the source model at any point during development. As a result, the accepted patches and case studies are precisely those that explain the source model's residual error profile, which may not align with the shared error profile across models. Through this mechanism, the RFR artifact accumulates model-specific content as it progresses through Phase 1 and Phase 2.

E Supplementary Audits

Protocol. The repaired rerun used the same item universe, CS-ICL cache, parser, prompt builder, retry path, scoring guard, and inactive passthrough rule as the locked protocol. It activated 39 RFCR+CS-ICL item-rule applications and had zero missing effect rows, baseline mismatches, parse errors, or failed calls (Table 10).

Audit field	Value
Main eligible items	400
Full18 items	1716
Activated applications	39
Missing effect rows	0
Baseline mismatches	0
Parse errors	0
Failed calls	0
Bootstrap samples	10,000

Table 10: Protocol audit for the repaired rerun.

AGIEval anchor diagnostic and rebuild check. AGIEval is excluded from the main claim because the available CS-ICL anchor underperforms raw prompting on all three tasks (Table 12, top). A CS-ICL anchor that is worse than raw prompting is not a valid starting point for a distillation improvement claim: any RFCR gain measured against it is at high risk of being confounded by the anchor deficit rather than attributable to the atoms. We therefore ran a development-only anchor rebuild check (E8) to determine whether a nonnegative anchor could be recovered for any task (Table 12, bottom). Two tasks (LogiQA and LSAT-LR) produced nonnegative development anchors, but LSAT-AR found no variant that satisfied the nonnegative-anchor requirement. As a result, AGIEval remains diagnostic and is not promoted to the main benchmark task set.

Leave-one-family-out proxy diagnostic. Table 13 evaluates whether the repaired atom set remains useful when a target model family is treated as held out during proxy gating. This is a retrospective diagnostic: the atom set is fixed from the source protocol and is not regenerated with each family excluded. It should therefore not be read as a full end-to-end leave-one-family-out experiment, but rather as a check on whether the accepted atoms are source-specific to the extent of actively hurting other families even under a favorable gate. The results show positive net transfer for Anthropic, Mistral, and Qwen, while Gemini incurs a net performance degradation. The performance degradation for Gemini is consistent with the cross-model result in Table 3 and reflects that the atom activation boundaries were derived from source-model failures without Gemini-specific validation.

Per-item mechanism examples. Table 14 gives examples of fixes, no-op passthrough, and boundary blocking. These examples are illustrative and are not used as additional evaluation evidence.

F Exploratory Holistic Rewrite Results

Iterative holistic rewrite is an exploratory alternative to atom appending. Rather than appending a routed atom alongside the CS-ICL anchor, holistic rewrite integrates the repair directly into the cheat sheet text, restructuring or merging rules where needed. It may help when a repair requires structural integration rather than a localized addition, but it is not yet robust enough to be a main contribution. Table 15 reports held-out accuracy for append versus holistic rewrite on causal judgment

Task	Status	n	CS-ICL	RFCR	Task	Status	n	CS-ICL	RFCR
boolean	no atom	100	89.00	89.00	causal judgement	no atom	87	66.67	66.67
date understanding	no atom	100	72.00	72.00	disambig. QA	main	100	83.00	87.00
formal fallacies	main	100	67.00	67.00	geometric shapes	main	100	56.00	63.00
LD five	no atom	100	73.00	73.00	LD seven	no atom	100	73.00	73.00
LD three	no atom	100	96.00	96.00	navigate	no atom	100	78.00	78.00
object counting	main	100	68.00	68.00	penguins table	no atom	58	74.14	74.14
colored objects	no atom	100	81.00	81.00	snarks	no atom	71	85.92	85.92
sports	no atom	100	96.00	96.00	temporal seq.	no atom	100	100.00	100.00
tracking shuffled	no atom	100	38.00	38.00	web of lies	excluded	100	65.00	65.00
Passthrough subtotal (14 tasks): n=1316, 77.74 / 77.74					Full 18 total: n=1716, 75.58 / 76.22				

Table 11: Full 18 task set: eligibility status and per-task accuracy (CS-ICL / RFCR). Tasks without an eligible atom are exact passthrough, so the two accuracies are identical by construction. Web of lies is excluded because its candidate atoms used activation boundary derived from test items, and no atom is deployed for it.

Task	n	Raw	CS-ICL	Net
LogiQA	300	57.67	57.36	−1
LSAT-AR	23	40.00	30.43	−11
LSAT-LR	373	81.57	79.22	−6
Aggregate	696	63.51	60.92	−18

Task	Raw dev	Anchor dev	Fix	Reg.
LogiQA	50.00	70.00	4	0
LSAT-AR	–	–	–	–
LSAT-LR	85.00	90.00	1	0

Table 12: Top: AGIEval rejected-anchor diagnostic. Bottom: development-only nonnegative-anchor rebuild check (E8).

Held-out family	CS-ICL	RFCR+CS	Fix	Reg.	Net
Anthropic	75.75	76.25	3	1	+2
Google/Gemini	64.25	63.75	3	5	−2
Mistral	37.25	40.75	14	0	+14
Qwen	51.00	53.75	13	2	+11

Table 13: Retrospective leave-one-family-out proxy-gating diagnostic. The atom set is frozen from the source protocol; this is not a full end-to-end LOO experiment. Positive net transfer holds for three of four families, with Gemini as the exception.

(CJ) and object counting (OC) across six target models (mean over seeds 1000/2000/3000).

On OC, holistic rewrite yields large gains for the two small open-weight models (`mistral-7b`: +11.7 pp, `qwen2.5-7b`: +24.0 pp) and modest gains for `claude-3.5-haiku` (+2.3 pp) and `llama-3.3-70b` (+1.7 pp), but substantially hurts `gpt-4.1` (−15.7 pp). On CJ, the picture is more mixed: holistic rewrite helps Gemini and Claude but hurts Mistral and is neutral on Qwen. The pattern suggests that holistic rewrite benefits models whose reasoning is less sensitive to prompt structure, particularly smaller open-weight models that may rely more on the integrated rule text. On the other hand, stronger proprietary models are harmed when the cheat sheet structure is disrupted. These results are promising for structured-integration use cases but should be treated as exploratory until replicated with tighter controls.

Task	ID	CS	RFCR	Gold	Atom	Blocked	Outcome
geo	0155	B	G	G	geo-6	no	fix
geo	0203	A	K	K	geo-1	no	fix
disambig	0171	C	B	B	disambig	no	fix
disambig	0242	C	A	A	disambig	no	fix
obj. count	0150	9	9	9	none	n/a	no-op
disambig	0154	A	A	A	disambig	yes	boundary no-op

Table 14: Qualitative per-item mechanism examples. The no-op rows illustrate inactive passthrough and boundary blocking rather than global prompt expansion.

	Causal judgment			Object counting		
Target model	App.	Hol.	Δ	App.	Hol.	Δ
`gpt-4.1`	**62.8**	61.3	−1.5	**83.7**	68.0	−15.7
`llama-3.3-70b`	**66.3**	64.4	−1.9	60.3	**62.0**	+1.7
`gemini-2.5-fl`	61.3	**65.9**	+4.6	**99.0**	97.0	−2.0
`claude-3.5-hk`	64.0	**65.1**	+1.1	89.7	**92.0**	+2.3
`mistral-7b`	**61.3**	54.8	−6.5	35.3	**47.0**	+11.7
`qwen2.5-7b`	57.9	57.9	0.0	36.3	**60.3**	+24.0

Table 15: Exploratory cross-model transfer accuracy (%, mean over seeds 1000/2000/3000) for append vs. holistic rewrite on causal judgment and object counting. Bold marks the better mode per cell; Δ = holistic − append (pp).